

PropManager — Roadmap Design Spec

Date: 2026-04-24

Status: Draft v1 — approved for implementation planning

Project: Convert the PropManager Vite/React mockup into a production-ready property management platform

Repository: <https://github.com/terekhovas/remix-of-dream-weaver-hub-13>

1. Overview

PropManager is a web application for **professional property management companies** operating in the **Greek market**. The company manages properties on behalf of landlords, handling tenancy agreements, rent collection, maintenance, and compliance.

The mockup is a fully designed React/TypeScript SPA with static data. This spec covers the roadmap to replace all static data with a live Supabase backend, add authentication, and extend the feature set across 6 milestones over 3 months.

2. Target Users

- **Property Managers (PMs):** Staff who manage day-to-day operations per property. Multiple PMs per company, each scoped to the company's full portfolio.
- **Company Admin:** Senior PM or head of PM who can invite team members and see the full portfolio dashboard.

Tenants and landlords are **not** end users — they are data subjects managed by the PMs.

3. Tech Stack

Layer	Technology	Notes
Frontend	Vite + React + TypeScript	Existing mockup — no framework migration
UI Components	shadcn/ui + Tailwind CSS	Already in mockup
Server State	React Query (TanStack Query)	Replace mockData.ts imports
Backend	Supabase (managed cloud)	Auth + PostgreSQL + Storage + Edge Functions

Layer	Technology	Notes
Database	PostgreSQL (via Supabase)	Row Level Security for multi-tenancy
File Storage	Supabase Storage	Tenancy docs, certificates, photos
Automation	Supabase Edge Functions (Deno)	Daily cron for arrears + contract expiry alerts
Email	Resend	Triggered from Edge Functions
Routing	React Router	Already in mockup

4. Architecture

Multi-tenancy Model

All data is scoped to a `company_id`. Row Level Security (RLS) policies on every table ensure PMs can only read and write rows belonging to their company. No backend proxy is needed — the Supabase JS client communicates directly from the browser.

Automation

A single daily Supabase Edge Function runs two checks: 1. **Arrears check:** flags unpaid rent at D1, D7, D30 and sends email alerts 2. **Contract expiry check:** emails the assigned PM 30 days before a management agreement expires

Document Storage

Documents are stored in **Supabase Storage** buckets. Google Drive URLs are also supported as text fields (for photos, signed contracts, passports) since the team already uses Drive for some document workflows. Both coexist: Supabase Storage for uploaded files, Drive URLs for externally hosted files.

5. Data Model

Core Tables

```
companies
  id, name, created_at
```

```
users
  id (= auth.users.id), company_id, full_name, role, created_at
```

```

properties
  id, company_id, manager_id → users
  address, city, postcode
  status: 'let' | 'vacant'
  -- Home tab physical attributes
  sqm, floor, bedrooms, bathrooms, wc
  year_built, year_renovated
  electricity_meter_no, water_meter_no
  photos_drive_url, google_maps_url
  furnishing_notes (text)
  -- Vacant tracking
  vacant_since_date

landlords
  id, company_id, property_id → properties
  name, email, phone, legal_rep_name, legal_rep_phone, legal_rep_email

management_agreements
  id, property_id → properties, company_id
  type: 'sublease' | 'management'
  subtype: 'guaranteed' | 'commission' | 'pm_collects' | 'owner_collects'
  guaranteed_rent_amount (nullable)
  commission_percent (nullable)
  management_fee_percent (nullable)
  start_date, end_date
  -- status is COMPUTED at query time: active / expiring_soon (30d) / expired
  -- do NOT store status; derive from end_date vs current date
  contract_drive_url
  document_id → documents (nullable)
  alert_30d_sent_at (nullable)

tenancies
  id, property_id, company_id
  tenant_name, tenant_email, tenant_phone
  passport_drive_url
  start_date, end_date
  monthly_rent, cpi_indexed
  bills_included: boolean
  rent_payment_day
  status: 'current' | 'future' | 'previous'
  agreement_drive_url

rent_payments
  id, tenancy_id, company_id
  month, year, amount
  paid: boolean

```

```

    paid_at (nullable)
    payment_reference
    proof_of_payment_drive_url (nullable)
    bank_statement_date (nullable)

arrears_log
    id, tenancy_id, company_id
    stage: 'D1' | 'D7' | 'D30'
    triggered_at
    email_sent: boolean
    resolved: boolean

documents
    id, property_id, company_id
    type: 'tenancy_agreement' | 'certificate_ydh' | 'certificate_pea' | 'checkin_report' | 'ch
    label, storage_path, uploaded_by → users, created_at

certificates
    id, property_id, company_id
    type: 'YDH' | 'PEA'
    -- status is COMPUTED at query time from valid_until vs current date
    -- valid: >60 days remaining | expiring: 60 days | expired: past date
    valid_until, rating (nullable)
    document_id → documents (nullable)

utilities
    id, property_id, company_id
    type: 'electricity' | 'water' | 'internet'
    provider, account_number
    meter_number (nullable)
    transfer_status: 'transferred' | 'not-transferred'
    transfer_date (nullable)
    transfer_proof_drive_url (nullable)
    monthly_amount

building_charges
    id, property_id, company_id
    manager_name, manager_phone
    annual_amount

utility_payments
    id, company_id
    utility_id → utilities (nullable)
    building_charge_id → building_charges (nullable)
    -- CHECK: exactly one of utility_id or building_charge_id must be non-null
    month, year, amount

```

```

    paid: boolean
    bill_drive_url (nullable)

comm_logs
    id, property_id, company_id
    type: 'whatsapp' | 'phone' | 'email' | 'in-person' | 'maintenance'
    summary, description, agreed_steps
    author_id → users, date

payouts
    id, property_id, company_id
    period_month, period_year
    gross_rent_collected
    deductions (JSON: fees, expenses)
    net_payout_amount
    paid_to_owner: boolean
    paid_at (nullable)
    bank_reference (nullable)

-- Vacant module
leads
    id, property_id, company_id
    type: 'agent' | 'individual'
    name, phone, email
    agent_company (nullable)
    status: 'new' | 'viewing_scheduled' | 'offer_made' | 'converted' | 'lost'
    comments, created_at

price_adjustments
    id, property_id, company_id
    old_price, new_price, reason, adjusted_at, adjusted_by → users

agents
    id, company_id
    name, company_name, phone, email
    area_specialty, commission_percent

-- Pipeline
pipeline_properties
    id, company_id
    address, city
    status: 'sourcing' | 'signed' | 'listed' | 'let'
    expected_rollout_date
    notes

```

6. Contract Types

Each property has exactly one active `management_agreement` record defining the business relationship:

A. Sub-lease Agreement

The company leases the property from the owner and sub-lets to a tenant.

Subtype	Description	Key Field
A1 — Guaranteed Income	Company pays owner fixed guaranteed rent regardless of vacancy	<code>guaranteed_rent_amount</code>
A2 — Commission	Company takes % of rent collected; owner bears vacancy risk	<code>commission_percent</code>

B. Management Agreement

Company manages on behalf of the owner, no sub-lease.

Subtype	Description	Key Field
B1 — PM Collects	Rent flows through PM account; PM deducts fee and remits to owner	<code>management_fee_percent</code>
B2 — Owner Collects	Owner receives rent directly; owner pays commission to PM	<code>commission_percent</code>

7. Tenant Traffic Light System

The traffic light status appears next to each tenant's name in the Properties list and Tenant tab:

Status	Colour	Condition
Good standing	Green	All payments paid on time for last 3 months
Warning	Yellow	1–6 days overdue on current month
Arrears	Red	7+ days overdue OR any D7/D30 arrears stage active

Computed at query time from `rent_payments` — no stored status field needed.

8. Arrears Automation (D1 / D7 / D30)

Daily Edge Function runs at 09:00 Athens time:

1. Queries all `rent_payments` where `paid = false` AND `month/year = current period` AND `payment_day` has passed.
 2. Calculates days overdue from `tenancy.rent_payment_day`.
 3. For each threshold crossed (1, 7, 30 days), inserts an `arrears_log` record and sends an email via Resend to the assigned PM — if that stage hasn't already been logged for this payment.
 4. D30 email includes a “legal notice flag” note to the PM.
-

9. Contract Expiry Alerts (30 days)

Same daily Edge Function also:

1. Queries `management_agreements` where `end_date = today + 30 days` AND `alert_30d_sent_at IS NULL`.
 2. Sends email to assigned PM with property address, contract type, and end date.
 3. Sets `alert_30d_sent_at = now()` to prevent duplicate sends.
 4. Dashboard shows a “**Contracts expiring in 30 days**” KPI card with a click-through to filtered property list.
-

10. Milestones

M1 — Auth & Supabase Setup (Weeks 1–2)

Deliverables: - Supabase project created, full schema migrated (all tables from §5) - RLS policies: all tables filtered by `company_id = auth.company_id` - Login / logout page in the React app - Protected route wrapper (redirect to login if not authenticated) - Company + user seed for testing (1 company, 2 PM accounts) - Multi-PM: invite flow (send invite email via Supabase Auth)

Exit criteria: Two PMs with separate logins can access the app, scoped to the same company's data.

M2 — Properties, Home Tab, Landlord Tab & Tenancies (Weeks 3–4)

Deliverables: - Properties list loaded from DB; columns: address, city, status, sqm, contract type, assigned PM - Add / Edit property form with all Home tab fields (sqm, floor, beds, baths, WC, year built/renovated, meter numbers, Drive photos URL, Google Maps URL, furnishing notes) - Landlord tab: owner contact form, legal rep fields, contract type selector, financial terms, Drive contract URL - `management_agreements` table wired to Landlord tab UI - Tenancy CRUD: add current tenancy, future tenancy, view previous tenancies - Tenant fields: name, email, phone, passport Drive URL, bills included/excluded, rent payment day - Property detail skeleton with tab routing (remaining tabs stubbed)

Exit criteria: PM can add a property with physical attributes, owner info, contract type, and an active tenancy.

M3 — Rent Roll, Dashboard & Utilities (Weeks 5–6)

Deliverables: - Rent Roll tab: multi-year grid loaded from `rent_payments` (live); click cell to mark paid/unpaid - CPI month highlighting based on `tenancy.start_date` anniversary - Tenant traffic light () in Properties list and Tenant tab, computed from `rent_payments` - Dashboard KPIs (all live): total properties, occupancy rate, rent collection %, arrears amount, monthly revenue, expiring contracts count - Arrears drill-down modal (live data) - Utilities tab: electricity/water/internet providers, meter numbers, account numbers, transfer status, building manager name + phone, building charges amount - Comm log tab: add/view communication log entries per property

Exit criteria: PMs can track rent payments live and see accurate portfolio health on the dashboard.

M4 — Documents, Arrears & Contract Expiry Automation (Weeks 7–8)

Deliverables: - Supabase Storage buckets: `documents`, `certificates` - Document upload UI: tenancy agreements, ΥΔΗ/ΠΕΑ certificates, check-in/out reports - Certificate status badges (valid / expiring / expired) based on `valid_until` date - Arrears Edge Function deployed: D1/D7/D30 email alerts via Resend, idempotent (no duplicate sends) - Contract expiry Edge Function deployed: 30-day email to PM, `alert_30d_sent_at` guard - `arrears_log` table records all triggered stages - Dashboard: “Contracts expiring in 30 days” KPI card

Exit criteria: System automatically emails PMs when rent is overdue or a management agreement is expiring.

M5 — Expenses, Rent Roll Proof of Payment & Payouts (Weeks 9–10)

Deliverables: - Expenses tab: register utility bills per property (amount, month, Drive URL); mark paid/unpaid; building charges register - `utility_payments` table wired to Expenses UI - Rent Roll: RF payment reference code per payment row (format TBD — pending Antonis); proof-of-payment Drive URL field; cell auto-turns green when URL is added; manual bank statement date field - Payouts tab: register owner payout per period, auto-calculate based on contract type (guaranteed rent / commission / management fee), mark as sent, bank reference field - Commission fee tracker for management agreements (B1/B2 contract types)

Open question before M5: Confirm RF payment code format with Antonis.

Exit criteria: PM can track all money flows — rent in, bills out, owner payouts, fees collected.

M6 — Vacant Module, Agents & Pipeline (Weeks 11–12)

Deliverables: - Vacant tab summary: Available / Leads / Viewings / New Contracts counters across portfolio - Per-property vacant detail: vacancy days counter, Spitogatos / XE.gr listing URLs, property description field, price adjustment log - Lead register: add lead (agent or individual), name/phone/email, status (new → viewing → offer → converted/lost), comments - Agents directory: name, company, phone, area specialty, commission % - Pipeline tab: list of new properties to onboard; fields: address, expected roll-out date, status (sourcing → signed → listed → let), notes

Exit criteria: Vacancy funnel is fully visible; new property pipeline tracked from sourcing through to live.

11. Out of Scope (V1)

The following features are **explicitly deferred** to V2 or later:

- WhatsApp Business API integration (two-way messaging)
- DocuSign / e-signature integration
- Spitogatos / XE.gr market data API (listing URLs captured manually in M6)
- Investment tab (purchase price, CapEx, IRR, yield calculations)
- Maintenance log module
- Periodical inspection module
- SMS alerts (Twilio) — email only for V1
- Property map (live, interactive)

- Mobile app

12. Open Questions

#	Question	Owner	Needed by
1	RF payment reference code format and issuance method	Antonis	Before M5
2	Document storage preference: Supabase Storage vs Google Drive API for direct upload	Sofia	Before M4

13. Key Decisions Log

Decision	Rationale
Keep Vite SPA (no Next.js migration)	Avoids migration cost; Supabase JS client works directly from browser; fits 3-month window
Supabase over custom backend	Eliminates backend server management; RLS handles multi-tenancy natively; Storage and Edge Functions cover all automation needs
Drive URLs as text fields (for now)	Team already uses Drive; simplest path for M1–M4; can migrate to Supabase Storage in V2
30-day single contract expiry alert	Chosen over 90/60/30 multi-stage to keep automation simple for V1
Traffic light computed at query time	Avoids stale status fields; always accurate against real rent_payments data
